

운영체제 Project 03 Wiki

한양대학교 데이터사이언스전공 2021052451 김태우

목차

- 운영체제 Project 03 Wiki
 - 목차
 - Large files
 - 과제 명세
 - Design
 - Implementation
 - Results

Large files

과제 명세

기존 xv6는 12개의 direct blocks와 1개의 singly-indirect block으로 이루어져 있어, $268\text{KB}(=1 \times 12 + 256 \times 1)$ 의 파일 크기 한계를 가졌다.

이 문제를 해결하기 위해 direct block을 1개 줄이고 doubly-indirect block을 도입하여, 파일 크기 한계를 $65,803\text{KB}(=1 \times 11 + 256 \times 1 + 65536 \times 1)$ 로 높이면서 기존 파일들과 호환성을 유지해야 한다.

이 과정에서 `FSSIZE` 등의 constant와, `bmap()` 등의 core function들을 수정하여야 한다.

Design

`param.h`에 선언된 `FSSIZE` 상수는 파일 시스템 최대 크기를 의미하며, 이 값을 200,000으로 수정하여야 한다.

`fs.h`에 선언된 `NDIRECT` 상수는 direct block의 개수이며, doubly-indirect block을 추가하기 위해 이 값을 12에서 11로 조절해야 한다.

`MAXFILE` 상수는 지원 가능한 최대 파일 크기를 의미한다.

`dinode`, `inode` 구조체 내부에 `addr[]` 배열 역시 수정해야 하며, `NDIRECT` 값이 1 감소한 것을 반영해야 한다.

`fs.c`의 `bmap()` 함수는 파일의 logical block number를 받아, disk block number를 반환하는 함수이다. 소스코드는 하단과 같으며, 코드 구조를 설명하는 주석을 추가하였다. 이 Project에서는 doubly-indirect block에 대응할 수 있게 수정해야 한다.

```
// Return the disk block address of the nth block in inode ip.
// If there is no such block, bmap allocates one.
// returns 0 if out of disk space.
static uint
bmap(struct inode *ip, uint bn)
{
    uint addr, *a;
    struct buf *bp;
```

```

if(bn < NDIRECT){ // logical block이 direct block 위에 있을 경우
    if((addr = ip->addrs[bn]) == 0){ // addr에 물리 주소 대입
        addr = balloc(ip->dev); // 할당되지 않은 경우
        if(addr == 0)
            return 0;
        ip->addrs[bn] = addr;
    }
    return addr;
}
bn -= NDIRECT;

if(bn < NINDIRECT){ // logical block이 singly-indirect block 위에 있을 경우
    // Load indirect block, allocating if necessary.
    // indirect block은 NDIRECT개의 block 다음에 존재함: addrs 배열의 NDIRECT 번째
    위치
    if((addr = ip->addrs[NDIRECT]) == 0){
        addr = balloc(ip->dev);
        if(addr == 0)
            return 0;
        ip->addrs[NDIRECT] = addr;
    }
    bp = bread(ip->dev, addr);
    a = (uint*)bp->data; // indirect block이 가리키는 위치
    if((addr = a[bn]) == 0){
        addr = balloc(ip->dev);
        if(addr){
            a[bn] = addr;
            log_write(bp);
        }
    }
    brelse(bp); // bread()에서 요청한 lock 해제
    return addr;
}

panic("bmap: out of range"); // 크기 초과
}

```

`itrunc` 함수는 `inode`가 가진 모든 데이터 block을 해제하며, 해당 `inode` 파일의 내용을 모두 비운다. 소스코드는 하단과 같으며, 코드 구조를 설명하는 주석을 추가하였다. 이 함수 역시 doubly-indirect block에 대응할 수 있게 수정해야 한다.

```

// Truncate inode (discard contents).
// Caller must hold ip->lock.
void
itrunc(struct inode *ip)
{
    int i, j;
    struct buf *bp;
    uint *a;

```

```

for(i = 0; i < NDIRECT; i++){ // direct block
    if(ip->addrs[i]){
        bfree(ip->dev, ip->addrs[i]);
        ip->addrs[i] = 0;
    }
}

if(ip->addrs[NDIRECT]){ // singly-indirect block
    bp = bread(ip->dev, ip->addrs[NDIRECT]);
    a = (uint*)bp->data; // indirect block이 가리키는 위치
    for(j = 0; j < NINDIRECT; j++){ // NINDIRECT개 block에 대해 free 실행
        if(a[j])
            bfree(ip->dev, a[j]);
    }
    brelse(bp);
    bfree(ip->dev, ip->addrs[NDIRECT]); // block free
    ip->addrs[NDIRECT] = 0;
}

ip->size = 0;
iupdate(ip);
}

```

Implementation

fs.h 파일을 다음과 같이 수정하였다.

```

#define NDIRECT 11
#define NINDIRECT (BSIZE / sizeof(uint))
#define NDINDIRECT ((BSIZE/sizeof(uint))*(BSIZE/sizeof(uint)))
#define MAXFILE (NDIRECT + NINDIRECT + NDINDIRECT)

// On-disk inode structure
struct dinode {
    short type;           // File type
    short major;          // Major device number (T_DEVICE only)
    short minor;          // Minor device number (T_DEVICE only)
    short nlink;          // Number of links to inode in file system
    uint size;            // Size of file (bytes)
    uint addrs[NDIRECT+2]; // Data block addresses
};

```

NDINDIRECT 변수는 1개의 doubly-indirect block에 저장 가능한 최대 크기를 의미하며, 이 값은 65536 이다.

MAXFILE 변수는 파일 최대 크기를 의미하며, 이 값을 계산하면 65803 이다.

NDIRECT는 1 감소하였으나 doubly-indirect block가 1개 추가되어, addrs 배열 크기를 NDIRECT+2 로 수정하였다. (file.h에 선언된 inode 구조체 역시 수정하였다.)

bmap() 함수에 추가한 부분은 다음과 같다. 구현한 부분에 대해 주석으로 설명하였다.

```
static uint
bmap(struct inode *ip, uint bn)
{
    uint addr, *a;
    struct buf *bp;
    struct buf *bp2; // doubly-indirect block 관련 필요

    if(bn < NDIRECT){
        (...)
    }
    bn -= NDIRECT;

    if(bn < NINDIRECT){
        (...)
    }
    bn -= NINDIRECT;

    if(bn < NDINDIRECT){ // logical block이 doubly-indirect block 위에 있을 경우
        // Load double-indirect block, allocating if necessary.
        if((addr = ip->addrs[NDIRECT+1]) == 0){ // doubly-indirect block가 위치한 주소
            addr = balloc(ip->dev);
            if(addr == 0)
                return 0;
            ip->addrs[NDIRECT+1] = addr;
        }
        bp = bread(ip->dev, addr);
        a = (uint*)bp->data;

        // doubly-indirect block 이 가리키는 block에서 몇 번째인지 찾기: 256으로 나눈 몫
        // bn >> 8 : indirect #
        if((addr = a[bn>>8]) == 0){
            addr = balloc(ip->dev);
            if(addr){
                a[bn>>8] = addr;
                log_write(bp);
            }
        }

        // singly-indirect block 내에서 몇 번째인지 찾기: 256으로 나눈 나머지
        // bn & 0xff(=255) : address #
        bp2 = bread(ip->dev, addr);
        a = (uint*)bp2->data;
        if((addr = a[bn & 0xff]) == 0){
            addr = balloc(ip->dev);
            if(addr){
                a[bn & 0xff] = addr;
                log_write(bp2);
            }
        }
    }

    brelse(bp2);
```

```

    brelse(bp);
    return addr;
}

panic("bmap: out of range");
}

```

`itrunc()` 함수에 추가한 부분은 다음과 같다. 구현한 부분에 대해 주석으로 설명하였다.

```

void
itrunc(struct inode *ip)
{
    // doubly-indirect block 구현을 위해 k, bp2, a2 추가
    int i, j, k;
    struct buf *bp;
    struct buf *bp2;
    uint *a;
    uint *a2;

    for(i = 0; i < NDIRECT; i++){ (...) }

    if(ip->addrs[NDIRECT]){ (...) }

    if(ip->addrs[NDIRECT+1]){ // doubly-indirect block 저장 위치
        bp = bread(ip->dev, ip->addrs[NDIRECT+1]);
        a = (uint*)bp->data;
        for(j = 0; j < NINDIRECT; j++){ // NDINDIRECT = NINDIRECT ^ 2
            if(a[j]){ // singly-indirect block
                bp2 = bread(ip->dev, a[j]);
                a2 = (uint*)bp2->data;
                for(k = 0; k < NINDIRECT; k++){ // nested loop
                    if(a2[k]) // address
                        bfree(ip->dev, a2[k]);
                }
                brelse(bp2);
                bfree(ip->dev, a[j]);
            }
        }
        brelse(bp);
        bfree(ip->dev, ip->addrs[NDIRECT+1]);
        ip->addrs[NDIRECT+1] = 0;
    }
    ip->size = 0;
    iupdate(ip);
}

```

test program 등록을 위한 `Makefile` 수정 내역은 생략하였다.

Results

```
xv6 kernel is booting
init: starting sh
$ bigfile
.....
.....
.....
.....
.....
.....
wrote 65803 blocks
bigfile done; ok
$
```

정상적으로 잘 실행되었으며, 실행에 20~30분 정도의 시간이 걸렸다.